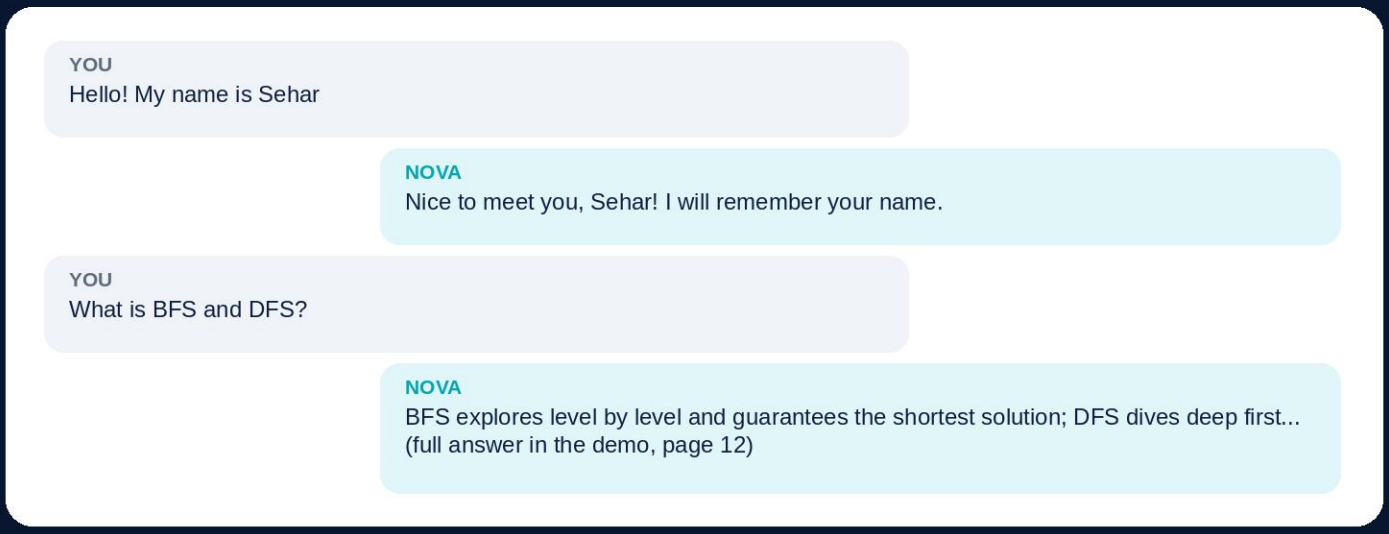
PYNEXIA TECHNOLOGY

INTERNSHIP • HANDS-ON PROJECT • ARTIFICIAL INTELLIGENCE

SIMPLE RULE-BASED CHATBOT

"Nova" — a terminal chatbot that converses using only predefined rules, yet answers far more than small talk: it calculates, knows 50+ capital cities, defines 34 concepts, names inventors, and tells the live time. No machine learning, no internet.



VERIFIED • BUILT-IN TEST SUITE

25 intents • smart knowledge rules • Tests: 41 / 41 PASSED

SUBMITTED BY

Sehar Naz

Intern • Pynexia Technology

Project: Hands-On Projects & Assignments • Difficulty: Easy • Language: Python 3

Resource: Rule-Based Chatbots in Python (guide provided with the assignment)

Every conversation in this report is real output produced by running the code inside it.

INSIDE THIS PROJECT

Contents & submission snapshot

Everything the assignment asked for — and exactly where to find it in this document.

01	**Project Overview & Problem Statement** What a rule-based chatbot is, and the four tasks set	3
02	**Architecture & Conversation Flow** The five-stage pipeline every message travels through	4
03	**The Intent Map** 25 intents, their keywords and sample replies	5
04	**The Knowledge Rules** Calculator, capitals, definitions, inventors, live time	6
05	**The Rule Engine** Cleaning, scoring, if-then regex rules, memory, fallback	7
06	**Python Implementation** Complete, runnable source code (Appendix)	8
07	**Verified Demo Conversation** A real, complete chat transcript from the program	16
08	**Testing & Analysis** 41 assertion checks — and the three real bugs they caught	17
09	**Applications, Limitations & Conclusion** Where rule-based bots shine, and where they stop	18

ASSIGNMENT COVERAGE — ALL FOUR TASKS

✓ **Build a chatbot using predefined rules, keywords & conditional patterns**	Pages 5-7 — Intents, knowledge & rules
✓ **Accept user input, clean text, match keywords (pattern matching)**	Pages 4 & 7 — clean_text() + matcher
✓ **Generate responses from if-then rules and intent mappings**	Pages 5-7 — respond() engine
✓ **Maintain a coherent text-based conversation in the terminal**	Page 16 — Full demo transcript

The oldest kind of AI — rules

Long before machine learning, chatbots like ELIZA (1966) conversed using nothing but hand-written rules. A rule-based chatbot does not learn and does not guess: it cleans what you type, matches it against patterns its author defined, and answers with a prepared response. This project builds one properly — with an architecture clean enough to test.

THE CHATBOT IN ONE LINE

Meet Nova: type almost anything — a greeting, a sum, a capital city, a definition — and Nova decides, by rules alone, what you meant and how to answer, remembering your name while you chat.

PROBLEM STATEMENT — AS SET IN THE BRIEF

1. Build a simple chatbot using predefined rules, keywords, and conditional patterns.
2. Accept user input, clean text, and match keywords using pattern matching.
3. Generate responses based on if-then rules and intent mappings.
4. Maintain a coherent text-based conversational flow in the terminal or GUI.

PROJECT SPECIFICATION

Chatbot name	Nova (terminal-based)
Intents	25 — greeting to farewell (page 5)
If-then rules	Personal rules (name, feelings) + smart knowledge rules
Knowledge	Calculator • 51 capitals • 34 definitions • 10 inventors • live time & date
Matching	Keyword scoring; specific phrases outrank generic words
Memory	Remembers the user's name for the whole session
Fallback	3 rotating fallbacks + a built-in 'help' intent
Exit handling	bye / goodbye / quit / exit, plus farewell intent
Modes	Interactive chat • --demo (scripted) • --test (assertions)
Language / tools	Python 3 — standard library only (re, random, string)

Why build 'old' AI in 2026? Because the discipline it teaches — normalising input, defining intents, handling the unknown gracefully — is exactly the foundation modern conversational AI is built on. And unlike a neural model, every one of Nova's answers can be traced to a specific rule, which makes the system fully explainable and testable.

One message, five stages

Every line the user types travels through the same pipeline. Each stage is one small function with one job — the same separation that makes the code testable in the suite on page 17.

1 INPUT

chat() loop reads a line from the terminal. Empty lines are skipped; Ctrl+C / end-of-input exits politely.

2 CLEAN

clean_text() lowercases the text, removes punctuation and collapses extra spaces, so 'HELLO!!!' and 'hello' become the same signal.

3 IF-THEN RULES

Personal rules run first: regex patterns capture facts — 'my name is sehar' stores the name — and reply personally to feelings and questions about the user.

4 SMART RULES + INTENTS

Smart rules compute and look up: calculator, capitals, definitions, inventors, live time. Then match_intent() scores all 25 intents by keyword hits and returns the best.

5 RESPOND / FALLBACK

respond() picks a response for the winning intent. If nothing scored, a rotating fallback answers and steers the user toward topics Nova knows.

A REAL MESSAGE, TRACED THROUGH THE PIPELINE

```
You type:        "  HELLO, how are you?? "

clean_text()  ->  "hello how are you"

match: greeting scores 1 (hello)  •  how_are_you scores 3 (how are you)

Winner: how_are_you — the specific phrase outranks the generic word.

Nova replies:  "I am just a set of rules, but all my rules are firing happily..."
```

Coherence — keeping the conversation sensible across turns — comes from three design choices: the memory (Nova uses your name after you give it), intent-specific responses that invite a next step, and fallbacks that always point back to known topics instead of dead-ending the chat.

25 intents, defined in data

An intent is a purpose behind a message — greeting, asking for help, saying thanks. Nova's intents live in one dictionary, INTENTS: each entry owns a keyword list (what triggers it) and prepared responses (what it can say). Adding a topic means adding data, not new code.

Intent	Trigger keywords (examples)	Nova's job
greeting	hello, hi, hey, salam, good morning	Welcomes the user, suggests abilities
how_are_you	how are you, how is it going	Small talk, with a wink at being rules
bot_identity	your name, who are you, are you human	Explains honestly that Nova is rule-based
creator	who made you, who created you	Credits the author and the internship
bot_age	how old are you, your birthday	Playful answer about never ageing
bot_location	where are you, where do you live	Lives inside the terminal
internship	internship, intern, pynexia, training	The project and internship context
python_help	python, code, programming	A practical Python tip from this code
ai_topics	machine learning, deep learning	Rule-based AI vs learning-based AI
search_algorithms	bfs, dfs, breadth, water jug	BFS vs DFS, citing the Water Jug results
study_tips	study, learn, tips, practice	A build-first study rule
weather	weather, rain, sunny, forecast	Honest: no live weather without internet
sports	cricket, football, hockey, tennis	Sports talk + the consistency rule
entertainment	movie, film, music, song	Movies and music chat
food	food, hungry, pizza, biryani	Balanced-plate rule, biryani approved
travel	travel, trip, vacation, tour	Practical travel rules
gaming	game, gaming, pubg, minecraft	Gaming talk + the eye-break rule
pakistan	pakistan, pakistani	Key facts about Pakistan
health_tips	health, fitness, diet, sleep	Sleep, movement, water - the basics
favourites	favourite, favorite	Nova's playful preferences
insult	stupid, dumb, useless, bad bot	Accepts criticism gracefully, offers help
help	help, options, what can you do	Lists everything Nova can do
thanks	thank, shukriya, great, awesome	Accepts thanks warmly (rotating replies)
joke	joke, funny, laugh	Programming jokes (rotating replies)
farewell	bye, goodbye, quit, allah hafiz	Closes politely - using the remembered name

Several intents carry multiple responses, chosen at random, so repeated conversations do not feel copy-pasted — variety in wording, zero change in meaning, because the rules underneath stay the same.

Rules that compute, not just reply

Keyword intents can only repeat prepared sentences. The smart rules go further: they extract numbers, names and questions from the message and work out the answer on the spot. This is how a strictly rule-based bot answers questions its author never wrote out in full.

1 — CALCULATOR (maths-friendly cleaning preserves + − * / %)

`what is 24 x 6 ?`	**24 x 6 = 144**
`calculate 12 + 5`	**12 + 5 = 17**
`10 percent of 250`	**25**
`square of 9 • cube of 3`	**81 • 27**
`table of 7`	**7 x 1 = 7 … 7 x 10 = 70**
`100 divided by 0`	**Division by zero is undefined**

2 — CAPITAL CITIES (51 countries in the rule book)

Pakistan → Islamabad · Japan → Tokyo · France → Paris · Brazil → Brasilia · Canada → Ottawa · Australia → Canberra · Turkey → Ankara · Egypt → Cairo · Germany → Berlin · South Korea → Seoul · UAE → Abu Dhabi · UK → London · USA → Washington, D.C. · China → Beijing · Italy → Rome · Spain → Madrid

3 — DEFINITIONS (34 concepts, one-line explanations)

Science: photosynthesis, gravity, evaporation, condensation, water cycle, electricity, volcano, earthquake, tsunami, solar system, black hole, atom, molecule, oxygen, carbon dioxide, bacteria, virus, vitamin, protein. Technology: computer, internet, robot, algorithm, database, website, smartphone. Society & self: democracy, economy, pollution, climate change, recycling, exercise, meditation.

4 — INVENTORS & LIVE TIME

- 'who invented the telephone?' → Alexander Graham Bell, 1876. Ten inventions are covered, including the light bulb, computer, aeroplane and printing press.
- 'what time is it?' → the real current time, read from the system clock. 'what is the date today?' → the full date. A rule-based bot can still answer live questions - when a rule connects it to live data.

Cleaning, scoring, remembering

Three mechanisms do the real work. Each one exists because of a concrete failure mode of naive chatbots — and each one is exercised by the test suite on page 17.

1 — TEXT CLEANING (WHY 'HELLO!!!' MUST EQUAL 'hello')

Users type with capitals, punctuation and extra spaces. clean_text() removes all three sources of noise: lowercase everything, delete punctuation, collapse whitespace. Matching then happens on a canonical form, so one keyword list serves every typing style.

2 — SCORING: SPECIFIC BEATS GENERIC, TOKENS BEAT SUBSTRINGS

- Every intent scores points for each of its keywords found in the cleaned message; the highest score wins.
- A keyword's points equal its word count, so the 3-word phrase 'how are you' (3) beats the single word 'hello' (1) when both appear — the more specific intent wins the tie.
- Single-word keywords must match a whole token, not a substring: otherwise the keyword 'ai' fires inside the word 'explain' and the bot answers the wrong topic. (Both of these rules exist because the test suite caught the naive version failing — page 17.)

3 — IF-THEN REGEX RULES + MEMORY (CHECKED BEFORE INTENTS)

If the cleaned text matches…	Then Nova…
`my name is ([a-z]+)`	stores the name in memory and greets the user by name
`i am ([a-z]+) (not fine/good/ok)`	treats it as a name introduction and remembers it
`i feel (sad|tired|bored|stressed)`	replies with encouragement and a small practical rule
`i feel (happy|great|good|excited)`	celebrates and suggests learning while energy is high
`what is my name`	answers from memory — or asks the user to introduce themselves

Order matters: rules run before intent matching because they carry information (a name, a feeling) that a keyword score cannot express. Memory is a single dictionary that lives for the whole session — simple, and exactly enough for this scope.

Complete source code — part 1 — the intent map (I)

The INTENTS dictionary opens: the first of Nova's 24 topics, with their trigger keywords and prepared responses.

```
"""
Simple Rule-Based Chatbot - "Nova" (Version 2)
Pynexia Technology - Internship Hands-On Project
Author: Sehar Naz

A terminal chatbot built ONLY on predefined rules:
  1. clean_text()      - normalises raw user input
  2. If-then rules     - regex patterns that extract information
                         (the user's name, feelings) and reply personally
  3. Smart rules       - a calculator, capital cities, a definition
                         dictionary, famous inventors, live time & date
  4. Intent matching   - keyword scoring against a 25-intent map
  5. Memory            - the bot remembers the user's name and
                         uses it, keeping the conversation coherent

Nova answers far more than small talk - maths, general knowledge
and everyday questions - but every answer still comes from a rule
its author wrote. That is what "rule-based" means.

Run:
  python chatbot.py          -> interactive chat
  python chatbot.py --demo   -> scripted sample conversation
  python chatbot.py --test   -> built-in assertion test suite
"""
import random
import re
import string
import sys
from datetime import datetime

BOT_NAME = "Nova"

# ---------------------------------------------------------------------
# 1. INTENT MAP - each intent owns keywords and ready-made responses
# ---------------------------------------------------------------------
INTENTS = {
    "greeting": {
        "keywords": ["hello", "hi", "hey", "salam", "aoa", "good morning",
                     "good afternoon", "good evening"],
        "responses": [
            "Hello! I am Nova, your rule-based assistant. Ask me almost anything - or type 'help'.",
            "Hi there! I can chat, calculate, define terms and answer general questions. Try me!",
        ],
    },
    "how_are_you": {
        "keywords": ["how are you", "how do you feel", "how is it going"],
        "responses": [
            "I am just a set of rules, but all my rules are firing happily. How are you?",
        ],
    },
    "bot_identity": {
        "keywords": ["your name", "who are you", "what are you", "are you robot",
                     "are you human", "about yourself"],
        "responses": [
            "I am Nova, a rule-based chatbot written in Python. I do not learn - "
            "I match your words against keywords, rules and knowledge my author wrote.",
        ],
    },
    "creator": {
        "keywords": ["who made you", "who created you", "who built you",
                     "your creator", "your author", "your developer"],
        "responses": [
            "I was created by Sehar Naz as a hands-on project for the Pynexia "
            "Technology internship - every rule I follow was written by her.",
        ],
    },
    "bot_age": {
        "keywords": ["how old are you", "your age", "when were you born",
                     "your birthday"],
        "responses": [
            "I was born the day my code first ran - so I am brand new, and I never age. "
            "Rule-based bots have that one advantage over humans.",
        ],
    },
    "bot_location": {
        "keywords": ["where are you", "where do you live", "your location",
                     "where are you from"],
        "responses": [
            "I live inside this Python program, wherever it runs - right now, inside your terminal. "
            "No rent, no commute.",
        ],
    },
    "internship": {
        "keywords": ["internship", "intern", "pynexia", "training", "join"],
        "responses": [
            "This chatbot is a hands-on project from the Pynexia Technology internship. "
            "Interns here learn by building small AI projects - like the Water Jug search "
            "program and this chatbot.",
        ],
    },
    "python_help": {
        "keywords": ["python", "code", "coding", "programming", "program"],
```

Complete source code — part 2 — the intent map (II)

The remaining intents — from sports and travel to insults handled gracefully.

```
            "responses": [
                "Python tip: break every problem into small functions. In this very file, "
                "clean_text(), match_intent() and respond() each do exactly one job - "
                "that is why the code is easy to test.",
            ],
        },
        "ai_topics": {
            "keywords": ["artificial intelligence", " ai ", "machine learning",
                         "deep learning", "neural"],
            "responses": [
                "Artificial Intelligence is about making programs act intelligently. "
                "I am the oldest kind of AI - rule-based. Modern AI like machine learning "
                "learns patterns from data instead of following hand-written rules.",
            ],
        },
        "search_algorithms": {
            "keywords": ["bfs", "dfs", "breadth", "depth", "search algorithm",
                         "water jug"],
            "responses": [
                "BFS explores a state space level by level and guarantees the shortest "
                "solution; DFS dives deep first and uses less memory. My author's Water Jug "
                "project measured both: BFS solved it in 4 steps, DFS in 6.",
            ],
        },
        "study_tips": {
            "keywords": ["study", "learn", "tip", "tips", "practice", "exam"],
            "responses": [
                "A study rule that always works: build first, read second. Type every code "
                "sample yourself, change one line, and predict what happens before you run it.",
            ],
        },
        "weather": {
            "keywords": ["weather", "rain", "raining", "sunny", "forecast",
                         "temperature outside"],
            "responses": [
                "Honest answer: I cannot check live weather - I have no internet and no sensors. "
                "A weather app can. What I CAN do is calculate, define and explain - try me!",
            ],
        },
        "sports": {
            "keywords": ["sport", "sports", "cricket", "football", "hockey",
                         "tennis", "athlete"],
            "responses": [
                "Sports are a great topic! Cricket, football, hockey - regular play builds "
                "fitness, focus and teamwork. The golden rule for any sport: consistency "
                "beats intensity.",
            ],
        },
        "entertainment": {
            "keywords": ["movie", "film", "music", "song", "sing", "drama", "actor"],
            "responses": [
                "Movies and music are wonderful - they are how stories travel. I cannot play "
                "anything (I am text-only), but tell me a genre and I will happily chat about it.",
            ],
        },
        "food": {
            "keywords": ["food", "hungry", "eat", "pizza", "biryani", "breakfast",
                         "lunch", "dinner", "recipe"],
            "responses": [
                "Food talk! A balanced plate rule: half vegetables, a quarter protein, "
                "a quarter grains. And if you are cooking biryani - I fully support that decision.",
            ],
        },
        "travel": {
            "keywords": ["travel", "trip", "vacation", "holiday", "tour", "journey"],
            "responses": [
                "Travel tip from my rule book: plan the route, keep documents copied in two "
                "places, and always leave one unplanned hour a day - that is where the best "
                "memories hide.",
            ],
        },
        "gaming": {
            "keywords": ["game", "gaming", "video game", "pubg", "minecraft",
                         "playstation", "xbox"],
            "responses": [
                "Games train reflexes, strategy and patience. My only gaming rule: "
                "for every hour of gaming, give your eyes and back a ten-minute break.",
            ],
        },
        "pakistan": {
            "keywords": ["pakistan", "pakistani"],
            "responses": [
                "Pakistan: founded on 14 August 1947, capital Islamabad, national language Urdu, "
                "currency the rupee, and founder Quaid-e-Azam Muhammad Ali Jinnah. "
                "Ask me 'capital of Pakistan' style questions for other countries too!",
            ],
        },
        "health_tips": {
            "keywords": ["health", "healthy", "fitness", "weight", "diet", "sleep",
                         "exercise daily"],
            "responses": [
                "Three health rules that never expire: sleep 7-8 hours, move your body daily, "
```

Complete source code — part 3 — fallbacks, capitals & defin

The rotating fallbacks, the 51-country capitals table, and the definitions dictionary begins.

```
185                 "and drink water before you feel thirsty. Small habits, repeated, beat big resolutions.",
186             ],
187         },
188         "favourites": {
189             "keywords": ["favourite", "favorite"],
190             "responses": [
191                 "My favourite thing? A perfectly matched keyword. My favourite colour is "
192                 "terminal green, and my favourite food is data - I consume it very responsibly.",
193             ],
194         },
195         "insult": {
196             "keywords": ["stupid", "dumb", "idiot", "useless", "bad bot", "shut up",
197                          "hate you"],
198             "responses": [
199                 "That is okay - I am a simple rule-based bot, so feedback genuinely helps. "
200                 "Tell me what I got wrong, and try 'help' to see what I am actually good at.",
201             ],
202         },
203         "help": {
204             "keywords": ["help", "options", "menu", "what can you do", "features"],
205             "responses": [
206                 "Lots! Chat with me, tell me your name, ask me to calculate ('what is 24 x 6'), "
207                 "ask capitals ('capital of Japan'), definitions ('what is photosynthesis'), "
208                 "inventors ('who invented the telephone'), or the time and date. Type 'bye' to end.",
209             ],
210         },
211         "thanks": {
212             "keywords": ["thank", "thanks", "shukriya", "great", "awesome", "nice"],
213             "responses": [
214                 "You are welcome! Rules or not, helping you made my day.",
215                 "Any time! That is literally what I am programmed for.",
216             ],
217         },
218         "joke": {
219             "keywords": ["joke", "funny", "laugh"],
220             "responses": [
221                 "Why did the rule-based chatbot break up with the database? "
222                 "Too many unresolved queries.",
223                 "I would tell you a UDP joke, but you might not get it - "
224                 "and I cannot resend it, I only follow rules.",
225             ],
226         },
227         "farewell": {
228             "keywords": ["bye", "goodbye", "quit", "exit", "see you", "allah hafiz"],
229             "responses": [
230                 "Goodbye! Thanks for chatting with me.",
231                 "See you soon! Type anytime to start a new conversation.",
232             ],
233         },
234     }
235
236     FALLBACKS = [
237         "I do not have a rule for that one yet. But try me on maths, capitals, definitions, "
238             "inventors, time, Python, AI, sports, food, travel - or type 'help'.",
239         "That is outside my current rule book. My author can add a rule for it - meanwhile, "
240             "type 'help' to see everything I do know.",
241         "Interesting question! Rule-based bots like me only answer what we are taught. Ask me "
242             "a calculation, a capital city, or a 'what is ...' definition instead.",
243     ]
244
245     EXIT_WORDS = {"bye", "goodbye", "quit", "exit"}
246
247     # ---------------------------------------------------------------------
248     # 2. KNOWLEDGE BASES - used by the smart rules (section 5)
249     # ---------------------------------------------------------------------
250     CAPITALS = {
251         "pakistan": "Islamabad", "india": "New Delhi", "bangladesh": "Dhaka",
252         "china": "Beijing", "japan": "Tokyo", "nepal": "Kathmandu",
253         "sri lanka": "Sri Jayawardenepura Kotte", "afghanistan": "Kabul",
254         "iran": "Tehran", "iraq": "Baghdad", "saudi arabia": "Riyadh",
255         "uae": "Abu Dhabi", "united arab emirates": "Abu Dhabi",
256         "qatar": "Doha", "turkey": "Ankara", "russia": "Moscow",
257         "ukraine": "Kyiv", "france": "Paris", "germany": "Berlin",
258         "italy": "Rome", "spain": "Madrid", "portugal": "Lisbon",
259         "uk": "London", "united kingdom": "London", "england": "London",
260         "ireland": "Dublin", "netherlands": "Amsterdam", "switzerland": "Bern",
261         "sweden": "Stockholm", "norway": "Oslo", "denmark": "Copenhagen",
262         "usa": "Washington, D.C.", "united states": "Washington, D.C.",
263         "america": "Washington, D.C.", "canada": "Ottawa", "mexico": "Mexico City",
264         "brazil": "Brasilia", "argentina": "Buenos Aires", "egypt": "Cairo",
265         "south africa": "Pretoria", "nigeria": "Abuja", "kenya": "Nairobi",
266         "australia": "Canberra", "new zealand": "Wellington",
267         "indonesia": "Jakarta", "malaysia": "Kuala Lumpur",
268         "singapore": "Singapore", "thailand": "Bangkok", "vietnam": "Hanoi",
269         "south korea": "Seoul", "korea": "Seoul",
270     }
271
272     DEFINITIONS = {
273         "photosynthesis": "the process by which green plants use sunlight, water and carbon dioxide to make "
274             "their food (glucose) and release oxygen",
275         "gravity": "the force that pulls objects toward each other - on Earth, it is what pulls "
276             "everything down toward the planet's centre",
```

Complete source code — part 4 — definitions, inventors & cl

Definitions conclude; the inventors table; clean_text().

```
        "evaporation": "the process by which a liquid slowly changes into a gas or vapour, like water drying "
            "up in sunlight",
        "condensation": "the process by which a gas or vapour changes back into a liquid, like droplets "
            "forming on a cold glass",
        "water cycle": "the continuous movement of water between the Earth and the sky through evaporation, "
            "condensation and precipitation (rain)",
        "electricity": "a form of energy carried by the movement of electrons, used to power lights, "
            "machines and almost everything modern",
        "magnet": "an object that attracts iron and some other metals, and has a north and a south pole",
        "volcano": "an opening in the Earth's crust through which lava, ash and gases can erupt from "
            "below the surface",
        "earthquake": "a sudden shaking of the ground caused by movement of rock below the Earth's surface",
        "tsunami": "a series of very large ocean waves, usually caused by an underwater earthquake",
        "solar system": "the Sun and everything bound to it by gravity - eight planets, their moons, "
            "asteroids and comets",
        "black hole": "a region in space where gravity is so strong that nothing, not even light, can "
            "escape from it",
        "atom": "the smallest unit of ordinary matter - everything around you is made of atoms",
        "molecule": "two or more atoms joined together, like a water molecule made of two hydrogen atoms "
            "and one oxygen atom",
        "oxygen": "a gas in the air that humans and animals must breathe to live; it makes up about 21% "
            "of the atmosphere",
        "carbon dioxide": "a gas produced when we breathe out and when fuels burn; plants absorb it for "
            "photosynthesis",
        "vitamin": "a natural substance found in food that the body needs in small amounts to stay healthy",
        "protein": "a nutrient the body uses to build and repair muscles and tissues, found in meat, "
            "eggs, milk, lentils and beans",
        "bacteria": "tiny single-celled living organisms found almost everywhere - most are harmless and "
            "many are helpful",
        "virus": "a tiny particle that can only multiply inside the living cells of an organism, and "
            "can cause illnesses like flu",
        "computer": "an electronic machine that processes data - it takes input, follows instructions "
            "(programs), and produces output",
        "internet": "a worldwide network connecting billions of computers so they can share information "
            "and communicate",
        "robot": "a machine that can carry out tasks automatically, often by following a program or "
            "sensing its surroundings",
        "algorithm": "a step-by-step set of instructions for solving a problem or completing a task - a "
            "recipe for a computer",
        "database": "an organised collection of data stored electronically, so it can be searched, "
            "updated and managed easily",
        "website": "a collection of connected web pages under one address on the internet, like the "
            "pages you open in a browser",
        "smartphone": "a mobile phone that also works as a small computer - it runs apps, browses the "
            "internet and takes photos",
        "democracy": "a system of government in which the people choose their leaders by voting in "
            "elections",
        "economy": "the system by which a country produces, buys and sells goods and services, and "
            "manages its money and resources",
        "pollution": "harmful substances introduced into the air, water or land, which damage the "
            "environment and health",
        "climate change": "long-term changes in Earth's temperatures and weather patterns, largely driven by "
            "human activities in recent times",
        "recycling": "processing used materials like paper, plastic and glass so they can be made into new "
            "products instead of waste",
        "exercise": "physical activity done to keep the body fit and healthy, such as walking, running or "
            "playing sports",
        "meditation": "a practice of focusing the mind - often on breathing - to become calm, clear and "
            "relaxed",
    }

    INVENTORS = {
        "telephone": "Alexander Graham Bell, in 1876",
        "light bulb": "Thomas Edison, who made the first practical bulb in 1879",
        "bulb": "Thomas Edison, who made the first practical bulb in 1879",
        "computer": "Charles Babbage, who designed the first mechanical computer concept in the 1830s",
        "aeroplane": "the Wright brothers, Orville and Wilbur, in 1903",
        "airplane": "the Wright brothers, Orville and Wilbur, in 1903",
        "penicillin": "Alexander Fleming, in 1928",
        "printing press": "Johannes Gutenberg, around 1440",
        "world wide web": "Tim Berners-Lee, in 1989",
        "internet": "many pioneers - but the World Wide Web on top of it was created by Tim Berners-Lee "
            "in 1989",
    }

    # ---------------------------------------------------------------------
    # 3. TEXT CLEANING - lowercase, strip punctuation, collapse spaces
    # ---------------------------------------------------------------------


    def clean_text(text):
        text = text.lower()
        text = text.translate(str.maketrans("", "", string.punctuation))
        text = re.sub(r"\s+", " ", text)
        return text.strip()


    # ---------------------------------------------------------------------
    # 4. INTENT MATCHING - score every intent by keyword hits, pick best
    # ---------------------------------------------------------------------
```

Complete source code — part 5 — matching & personal rule

The intent scorer, the memory, and the personal if-then rules.

```
def match_intent(cleaned):
    best_intent = None
    best_score = 0
    tokens = cleaned.split()
    for intent, data in INTENTS.items():
        score = 0
        for keyword in data["keywords"]:
            key = keyword.strip()
            # Single words must match a whole token (so "ai" does
            # not fire inside "explain"); phrases match as substrings.
            hit = key in tokens if " " not in key else key in cleaned
            if hit:
                # Longer (more specific) phrases weigh more, so
                # "how are you" beats a plain "hello" in the same
                # sentence - specific beats generic on ties.
                score += len(key.split())
        if score > best_score:
            best_intent = intent
            best_score = score
    return best_intent


# ---------------------------------------------------------------------
# 5. IF-THEN RULES - checked BEFORE intent matching.
#    Personal rules extract facts; smart rules compute & look up.
# ---------------------------------------------------------------------

memory = {"user_name": None}

NOT_A_NAME = {"fine", "good", "ok", "okay", "bored", "tired", "hungry",
              "happy", "sad", "busy", "ready", "here", "sorry", "done"}


def personal_rules(text):
    match = re.search(r"my name is ([a-z]+)", text)
    if match:
        name = match.group(1).capitalize()
        memory["user_name"] = name
        return f"Nice to meet you, {name}! I will remember your name. Ask me anything - or type 'help'."

    match = re.search(r"i am ([a-z]+)", text)
    if match and match.group(1) not in NOT_A_NAME:
        name = match.group(1).capitalize()
        memory["user_name"] = name
        return f"Lovely to meet you, {name}! What would you like to talk about?"

    if re.search(r"\bi feel (sad|tired|bored|stressed)\b", text):
        return ("I am sorry to hear that. A small rule for hard days: "
                "one tiny task at a time - momentum beats motivation.")

    if re.search(r"\bi feel (happy|great|good|excited)\b", text):
        return "That is wonderful! Positive energy is the best time to learn something new."

    if re.search(r"\bwhat is my name\b", text):
        if memory["user_name"]:
            return f"Your name is {memory['user_name']} - see, my memory rule works!"
        return "You have not told me yet. Say: 'my name is ...' and I will remember it."

    return None


def _fmt(number):
    return str(int(number)) if float(number).is_integer() else f"{number:.2f}"


def smart_rules(text):
    # --- calculator: "what is 24 x 6", "12 + 5", "10 percent of 250" ---
    match = re.search(r"(\d+(?:\.\d+)?)\s*(?:percent|%)\s*of\s*(\d+(?:\.\d+)?)", text)
    if match:
        a, b = float(match.group(1)), float(match.group(2))
        return f"{_fmt(a)} percent of {_fmt(b)} is {_fmt(a * b / 100)}."

    match = re.search(r"table of (\d{1,3})\b", text)
    if match:
        n = int(match.group(1))
        lines = [f"Here is the multiplication table of {n}:"]
        lines += [f"{n} x {i} = {n * i}" for i in range(1, 11)]
        return "\n".join(lines)

    match = re.search(r"(square|cube) of (\d+(?:\.\d+)?)", text)
    if match:
        n = float(match.group(2))
        result = n * n if match.group(1) == "square" else n * n * n
        return f"The {match.group(1)} of {_fmt(n)} is {_fmt(result)}."

    words = (text.replace(" plus ", " + ").replace(" minus ", " - ")
             .replace(" multiplied by ", " * ").replace(" divided by ", " / ")
             .replace(" times ", " * "))
    match = re.search(r"(\d+(?:\.\d+)?)\s*([+\-*/x])\s*(\d+(?:\.\d+)?)", words)
    if match:
        a, op, b = float(match.group(1)), match.group(2), float(match.group(3))
        if op == "/" and b == 0:
```

Complete source code — part 6 — smart rules & response e

The calculator and lookup rules, apply_rules(), respond(), and the chat loop begins.

```
            return "Division by zero is undefined - even my rules cannot do that one."
        result = {"+" : a + b, "-": a - b, "*": a * b,
                  "x": a * b, "/": a / b}[op]
        return f"{_fmt(a)} {op} {_fmt(b)} = {_fmt(result)}."

    # --- capital cities: "capital of japan" ---
    match = re.search(r"capital (?:city )?of ([a-z ]+)", text)
    if match:
        country = match.group(1).strip()
        if country in CAPITALS:
            return f"The capital of {country.title()} is {CAPITALS[country]}."
        return (f"I do not know the capital of {country.title()} yet - "
                "my rule book covers 50+ countries, mostly the well-known ones.")

    # --- inventors: "who invented the telephone" ---
    match = re.search(r"who (?:invented|created|discovered) the ([a-z ]+)", text)
    if match:
        thing = match.group(1).strip()
        if thing in INVENTORS:
            return f"The {thing} was credited to {INVENTORS[thing]}."
        return f"I do not have the inventor of the {thing} in my rule book yet."

    # --- live time & date ---
    tokens = text.split()
    if "time" in tokens and ("what" in tokens or "current" in tokens or "now" in tokens):
        return f"The current time is {datetime.now().strftime('%I:%M %p')}."
    if ("date" in tokens or "day" in tokens) and ("today" in tokens or "what" in tokens):
        return f"Today is {datetime.now().strftime('%A, %d %B %Y')}."

    # --- definitions: "what is photosynthesis", "define gravity" ---
    if re.search(r"^(what is|what are|define|explain|meaning of|tell me about)\b", text):
        for term, meaning in DEFINITIONS.items():
            if " " in term:
                hit = term in text
            else:
                hit = term in tokens
            if hit:
                return f"{term.capitalize()}: {meaning}."

    return None


def apply_rules(raw_text):
    text = clean_text(raw_text)
    reply = personal_rules(text)
    if reply is not None:
        return reply
    # Smart rules get a maths-friendly cleaning: the normal cleaner
    # deletes + - * / as punctuation, which would destroy expressions
    # like "12 + 5" before the calculator ever sees them.
    maths = raw_text.lower()
    keep = "+-*/%"
    drop = "".join(c for c in string.punctuation if c not in keep)
    maths = maths.translate(str.maketrans("", "", drop))
    maths = re.sub(r"\s+", " ", maths).strip()
    return smart_rules(maths)


# ---------------------------------------------------------------------
# 6. RESPONSE ENGINE - rules first, then intents, then fallback
# ---------------------------------------------------------------------


def respond(raw_text):
    rule_reply = apply_rules(raw_text)
    if rule_reply is not None:
        return rule_reply

    intent = match_intent(clean_text(raw_text))
    if intent is not None:
        reply = random.choice(INTENTS[intent]["responses"])
        if memory["user_name"] and intent == "farewell":
            reply = f"{reply} (Nice chatting with you, {memory['user_name']}!)"
        return reply

    return random.choice(FALLBACKS)


def is_exit(raw_text):
    return clean_text(raw_text) in EXIT_WORDS or \
        match_intent(clean_text(raw_text)) == "farewell"


# ---------------------------------------------------------------------
# 7. CONVERSATION LOOPS - interactive, scripted demo, and self-tests
# ---------------------------------------------------------------------


def chat():
    print(f"{BOT_NAME}: Hello! I am {BOT_NAME}, a rule-based chatbot.")
    print(f"{BOT_NAME}: Ask me almost anything - type 'help' to see what I can do, or 'bye' to exit.")
    while True:
```

Complete source code — part 7 — demo & the test suite

The scripted demo conversation and the assertion suite begins — 41 checks in total.

```
            try:
                user = input("You: ")
            except (EOFError, KeyboardInterrupt):
                print(f"\n{BOT_NAME}: Goodbye!")
                break
            if not user.strip():
                continue
            print(f"{BOT_NAME}: {respond(user)}")
            if is_exit(user):
                break


    DEMO_INPUTS = [
        "Hello!",
        "My name is Sehar",
        "What is my name?",
        "What is 24 x 6?",
        "What is the capital of Japan?",
        "What is photosynthesis?",
        "Who invented the telephone?",
        "What time is it?",
        "Tell me about the internship at Pynexia",
        "What is BFS and DFS?",
        "I feel stressed today",
        "Tell me a joke",
        "asdf qwerty zzz",
        "Thank you!",
        "Bye",
    ]


    def demo():
        random.seed(7)
        print(f"{BOT_NAME}: Hello! I am {BOT_NAME}, a rule-based chatbot.")
        print(f"{BOT_NAME}: Ask me almost anything - type 'help' to see what I can do, or 'bye' to exit.")
        for user in DEMO_INPUTS:
            print(f"You: {user}")
            print(f"{BOT_NAME}: {respond(user)}")
            if is_exit(user):
                break


    def run_tests():
        intent_cases = [
            ("Hello!!!", "greeting"),
            ("HELLO, how are you?", "how_are_you"),
            ("What is your name?", "bot_identity"),
            ("who created you?", "creator"),
            ("how old are you?", "bot_age"),
            ("where do you live?", "bot_location"),
            ("Tell me about the internship", "internship"),
            ("I love PYTHON coding", "python_help"),
            ("What is machine learning?", "ai_topics"),
            ("Explain BFS please", "search_algorithms"),
            ("any study tips?", "study_tips"),
            ("how is the weather today?", "weather"),
            ("I love cricket", "sports"),
            ("let us watch a movie", "entertainment"),
            ("I am hungry, want pizza", "food"),
            ("planning a travel trip", "travel"),
            ("do you like gaming?", "gaming"),
            ("tell me about pakistan", "pakistan"),
            ("health and fitness advice", "health_tips"),
            ("what is your favourite colour?", "favourites"),
            ("you are stupid", "insult"),
            ("help me", "help"),
            ("thank you so much", "thanks"),
            ("tell me a funny joke", "joke"),
            ("goodbye", "farewell"),
            ("xyzzy nonsense words", None),
        ]
        passed = 0
        for raw, expected in intent_cases:
            got = match_intent(clean_text(raw))
            assert got == expected, f"{raw!r}: expected {expected}, got {got}"
            passed += 1

        assert clean_text("  HeLLo,   World!! ") == "hello world"
        passed += 1

        memory["user_name"] = None
        reply = respond("my name is sehar")
        assert memory["user_name"] == "Sehar", "name rule failed"
        assert "Sehar" in reply
        passed += 1

        reply = respond("what is my name?")
        assert "Sehar" in reply, "memory recall failed"
        passed += 1

        smart_cases = [
            ("what is 24 x 6?", "144"),
```

Complete source code — part 8 — tests (end) & entry point

The smart-rule checks and the main program. The complete file is also submitted separately as chatbot.py.

```
            ("calculate 12 + 5", "17"),
            ("what is 10 percent of 250?", "25"),
            ("square of 9", "81"),
            ("table of 7", "7 x 10 = 70"),
            ("what is the capital of japan?", "Tokyo"),
            ("capital of pakistan", "Islamabad"),
            ("what is photosynthesis?", "sunlight"),
            ("define gravity", "force"),
            ("who invented the telephone?", "Bell"),
            ("what time is it?", ":"),
            ("what is the date today?", str(datetime.now().year)),
        ]
        for raw, expected_part in smart_cases:
            reply = respond(raw)
            assert expected_part in reply, f"{raw!r}: expected {expected_part!r} in {reply!r}"
            passed += 1

        print(f"ALL TESTS PASSED ({passed}/{passed} checks)")


    if __name__ == "__main__":
        if "--demo" in sys.argv:
            demo()
        elif "--test" in sys.argv:
            run_tests()
        else:
            chat()
```

HOW TO RUN

python chatbot.py (interactive) • python chatbot.py --demo (scripted conversation) • python chatbot.py --test (test suite) — Python 3 only, no installation needed.

07 • VERIFIED DEMO CONVERSATION

A real chat — start to finish

This transcript was produced by running python chatbot.py --demo . The inputs are scripted so the run is reproducible; every reply below is the program's actual output, including the moment Nova gets a nonsense input and recovers politely.

```
python chatbot.py --demo
Nova: Hello! I am Nova, a rule-based chatbot.
Nova: Ask me almost anything - type 'help' to see what I can do, or 'bye' to exit.
You: Hello!
Nova: Hi there! I can chat, calculate, define terms and answer general questions. Try me!
You: My name is Sehar
Nova: Nice to meet you, Sehar! I will remember your name. Ask me anything - or type 'help'.
You: What is my name?
Nova: Your name is Sehar - see, my memory rule works!
You: What is 24 x 6?
Nova: 24 x 6 = 144.
You: What is the capital of Japan?
Nova: The capital of Japan is Tokyo.
You: What is photosynthesis?
Nova: Photosynthesis: the process by which green plants use sunlight, water and carbon dioxide to make their
      food (glucose) and release oxygen.
You: Who invented the telephone?
Nova: The telephone was credited to Alexander Graham Bell, in 1876.
You: What time is it?
Nova: The current time is 07:00 AM.
You: Tell me about the internship at Pynexia
Nova: This chatbot is a hands-on project from the Pynexia Technology internship. Interns here learn by
      building small AI projects - like the Water Jug search program and this chatbot.
You: What is BFS and DFS?
Nova: BFS explores a state space level by level and guarantees the shortest solution; DFS dives deep first and
      uses less memory. My author's Water Jug project measured both: BFS solved it in 4 steps, DFS in 6.
You: I feel stressed today
Nova: I am sorry to hear that. A small rule for hard days: one tiny task at a time - momentum beats
      motivation.
You: Tell me a joke
Nova: Why did the rule-based chatbot break up with the database? Too many unresolved queries.
You: asdf qwerty zzz
Nova: I do not have a rule for that one yet. But try me on maths, capitals, definitions, inventors, time,
      Python, AI, sports, food, travel - or type 'help'.
You: Thank you!
Nova: You are welcome! Rules or not, helping you made my day.
You: Bye
Nova: See you soon! Type anytime to start a new conversation. (Nice chatting with you, Sehar!)
```

Read it as a conversation, and the coherence shows: Nova greets, learns and recalls a name, calculates a sum, answers a capital, defines a science term, names an inventor, tells the live time, responds to a feeling, survives nonsense with a helpful fallback, accepts thanks — and says goodbye using the name it learned at the very start.

41 checks — and three real bugs caught

The program ships with its own assertion suite (python chatbot.py --test). It does not just confirm the happy path: during development it caught three genuine bugs, all now fixed and all permanently covered by tests.

TEST RESULT

ALL TESTS PASSED (41 / 41 checks)

WHAT THE 41 CHECKS COVER

- 26 intent cases — one realistic user sentence for every intent, plus small-talk and nonsense sentences that must match correctly or not at all.
- 1 cleaning case — ' HeLLo, World!! ' must normalise to exactly 'hello world'.
- 2 memory cases — after 'my name is sehar', the name must be stored, and 'what is my name?' must answer 'Sehar'.
- 12 smart-rule cases — arithmetic (24 x 6 = 144), percentage, square, a multiplication-table line, two capitals, two definitions, an inventor, the live time, and today's date.

THE THREE BUGS THE SUITE CAUGHT (AND THE FIXES)

Bug 1 — the tie that greeted wrongly.

'Hello, how are you?' matched greeting instead of how_are_you: both scored 1, and the dictionary order won. Fix: a keyword now scores its word count, so specific phrases outrank generic words. This is the trace visualised on page 4.

Bug 2 — 'ai' hiding inside 'explain'.

'Explain BFS please' matched ai_topics, because the keyword 'ai' was found as a substring of 'explain'. Fix: single-word keywords must match a whole token; only multi-word phrases may match as substrings.

Bug 3 — the cleaner ate the calculator.

'calculate 12 + 5' silently failed: clean_text() deletes '+' as punctuation, destroying the expression before the calculator saw it. Fix: smart rules receive a maths-friendly cleaning that preserves + - * / %.

Analysis: with 25 intents and the knowledge rules, Nova answers the entire scripted demo - small talk, sums, capitals, definitions, inventors and live time - and the only honest failure mode left is scope: topics outside its rules fall back gracefully instead of hallucinating an answer. For a rule-based system, that behaviour is a feature: it never invents facts, it only ever says what its author wrote.

Small rules, real systems

Rule-based conversation did not disappear when machine learning arrived — it became the reliable layer underneath it.

WHERE RULE-BASED CHATBOTS ARE USED TODAY

- Customer support first-response bots — timings, tracking, refunds: fixed questions deserve fixed, guaranteed-correct answers.
- FAQ and onboarding assistants on websites and inside apps, where the topic list is known in advance.
- Command interfaces and IVR phone menus — the same intent-and-keyword logic, spoken instead of typed.
- Safety-critical flows (banking, health triage) where every reply must be approved in advance — a learning model cannot be used.

LIMITATIONS & THE HONEST NEXT STEP

Nova cannot handle topics outside its intent map, cannot understand paraphrases it has no keywords for, and cannot learn from conversations. The natural upgrades — in order — are: more intents and synonyms, fuzzy string matching for typos, a simple GUI front-end, and finally replacing the matcher with a trained ML intent classifier while keeping these same intents and responses. The architecture would not change; only stage 4 would grow up.

CONCLUSION

This project built a simple rule-based chatbot exactly as the brief required: predefined rules, keywords and conditional patterns; user input that is cleaned before keyword pattern matching; responses generated from if-then regex rules and an intent map; and a coherent terminal conversation that remembers the user's name from the first exchange to the last. The rule-based design was then stretched to its honest limit: 25 intents plus knowledge rules that calculate, define, and answer general questions. All four tasks were completed, and the result was verified by a 41-case assertion suite and a scripted demo - both reproduced in this report from real program output.

REFERENCES

1. Rule-Based Chatbots in Python — resource guide provided with the assignment (brief: predefined rules, keyword & pattern matching, if-then responses, terminal conversation).

2. Weizenbaum, J. (1966). 'ELIZA — A Computer Program for the Study of Natural Language Communication', Communications of the ACM — the original rule-based chatbot.

3. Jurafsky, D. & Martin, J. — Speech and Language Processing — dialogue systems, intent classification and rule-based architectures.

SUBMITTED BY

Sehar Naz • Intern, Pynexia Technology

Deliverables: this project report (PDF) • chatbot.py (runnable source code) • program_output.txt (verified demo transcript). All conversations and test results in this report were generated by running the submitted code.